

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    char command[100];
```

```
    printf("Enter a Linux command: ");
```

```
    scanf("%s", command);
```

```
    pid_t pid = fork();
```

```
    if (pid < 0)
```

```
    {
```

```
        perror("fork failed");
```

```
        return 1;
```

```
    }
```

```
    else if (pid == 0)
```

```
{  
    printf("Child Process\n");  
    printf("Child PID: %d\n", getpid());  
    printf("Parent PID: %d\n", getppid());  
  
    execlp(command, command, NULL);  
  
    perror("exec failed");  
    exit(1);  
}  
else  
{  
    printf("Parent Process\n");  
    printf("Parent PID: %d\n", getpid());  
    printf("Child PID: %d\n", pid);  
  
    wait(NULL);  
  
    printf("Child process completed.\n");  
}  
  
return 0;  
}
```

```

gopisetti_akshay@AKSHAY:~$ gcc exp1.c -o exp1
gopisetti_akshay@AKSHAY:~$ ./exp1
Enter a Linux command: ls
Parent Process
Parent PID: 470
Child PID: 471
Child Process
Child PID: 471
Parent PID: 470
OSLab      exp2.c  fork1.c  hello0.c  hello4.c  no_zombie  read.c    wait.c    zombie.c
OSSP       exp3.c  fork_exec  hello1.c  hello5.c  no_zombie.c  sample    wait_pid  zombie_ele
close      exp3.c  fork_exec.c  hello1.c  hello5.c  open       sample.c  wait_pid.c  zombie_ele.c
close.c    exp4.c  fork_exec1  hello2.c  hello6.c  open.c     sample.txt  wait_pid1  zombie_ele.c
destination.txt exp4.c  fork_exec1.c  hello2.c  hello6.c  output.txt  shell      wait_pid1.c
exp1       file    hello     hello3.c  hello7.c  pc_fork    shell.c    write     write.c
exp1.c     file.c  hello.c   hello3.c  hello7.c  pc_fork.c  source.txt  write.c
exp2       fork1   hello0    hello4    input.txt  read       wait       zombie
Child process completed.
gopisetti_akshay@AKSHAY:~$ |

```

2. Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

```
#include <stdio.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int source, destination;
```

```
    char buffer[1024];
```

```
    int bytes;
```

```
    source = open("source.txt", O_RDONLY);
```

```
    if (source < 0)
```

```
    {
```

```
        perror("Error opening source file");
```

```

        return 1;
    }

    destination = open("destination.txt",
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

    if (destination < 0)
    {
        perror("Error creating destination file");
        close(source);
        return 1;
    }

    while ((bytes = read(source, buffer, sizeof(buffer))) > 0)
    {
        write(destination, buffer, bytes);
    }

    close(source);
    close(destination);

    printf("File copied successfully.\n");

    return 0;
}

```

```

File copied successfully.
gopisetti_akshay@AKSHAY:~$ nano exp2.c
gopisetti_akshay@AKSHAY:~$ echo "Hello, this is a test file." > source.txt
gopisetti_akshay@AKSHAY:~$ gcc exp2.c -o exp2
gopisetti_akshay@AKSHAY:~$ ./exp2
File copied successfully.

```

```
File copied successfully.  
gopisetti_akshay@AKSHAY:~$ cat destination.txt  
Hello, this is a test file.  
gopisetti_akshay@AKSHAY:~$ echo "Hello Ubuntu" > sample.txt  
gopisetti_akshay@AKSHAY:~$ strace cat sample.txt
```

3. Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    pid_t pid;
```

```
    printf("Before fork()\n");
```

```
    printf("Process PID: %d\n", getpid());
```

```
    pid = fork();
```

```
    if (pid < 0)
```

```
    {
```

```
        perror("fork failed");
```

```
        return 1;
```

```
    }
```

```
    if (pid == 0)
```

```
    {
```

```
    printf("\nChild Process\n");
    printf("PID : %d\n", getpid());
    printf("PPID : %d\n", getppid());

    sleep(10);

    printf("Child process terminating...\n");
}
else
{
    printf("\nParent Process\n");
    printf("PID : %d\n", getpid());
    printf("PPID : %d\n", getppid());

    sleep(5);

    printf("Parent process terminating...\n");
}

return 0;
}
```

```

gopisetti_akshay@AKSHAY:~$ gcc exp3.c -o exp3
gopisetti_akshay@AKSHAY:~$ ./exp3
Before fork()
Process PID: 631

Parent Process
PID : 631
PPID : 331

Child Process
PID : 632
PPID : 631
Parent process terminating...
gopisetti_akshay@AKSHAY:~$ Child process terminating...
ps -ef
UID      PID    PPID  C STIME TTY          TIME CMD
root      1        0  0 15:17 ?           00:00:00 /sbin/init
root      2        1  0 15:17 hvc0       00:00:00 /init
root      6        2  0 15:17 hvc0       00:00:00 plan9 --control-socket 7
root     43        1  0 15:17 ?           00:00:00 /usr/lib/systemd/systemd
systemd+  78        1  0 15:17 ?           00:00:00 /usr/lib/systemd/systemd
root     84        1  0 15:17 ?           00:00:00 /usr/lib/systemd/systemd
root    104        1  0 15:17 ?           00:00:00 /bin/sh /usr/lib/systemd
root    105        1  0 15:17 ?           00:00:00 /usr/sbin/cron -f -P
message+ 106        1  0 15:17 ?           00:00:00 @dbus-daemon --system --
root    122        1  0 15:17 ?           00:00:00 /usr/bin/python3 /usr/bi
root    163        1  0 15:17 ?           00:00:00 /usr/lib/systemd/systemd
root    166        1  0 15:17 ?           00:00:00 /usr/libexec/wsl-pro-ser
syslog   186        1  0 15:17 ?           00:00:00 /usr/sbin/rsyslogd -n -i
root    207        1  0 15:17 tty1       00:00:00 /usr/sbin/agetty --nores
_chrony  216       104  0 15:17 ?           00:00:00 /usr/sbin/chronyd -n -F
_chrony  232       216  0 15:17 ?           00:00:00 /usr/sbin/chronyd -n -F
root    254        1  0 15:17 ?           00:00:00 /usr/bin/python3 /usr/sh
root    329        2  0 15:17 ?           00:00:00 /init
root    330       329  0 15:17 ?           00:00:00 /init
gopiset+ 331       330  0 15:17 pts/0      00:00:00 -bash
root    332        2  0 15:17 ?           00:00:00 login -- gopisetti_aksha
gopiset+ 374        1  0 15:17 ?           00:00:00 /usr/lib/systemd/systemd
gopiset+ 376       374  0 15:17 ?           00:00:00 (sd-pam)
gopiset+ 402       332  0 15:17 pts/1      00:00:00 -bash
gopiset+ 642       331  0 15:22 pts/0      00:00:00 ps -ef

```

```

gopisetti_akshay@AKSHAY:~$ ps -el
F S      UID      PID    PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S      0         1        0  0  80   0 - 6063 -      ?           00:00:00 system
5 S      0         2        1  0  80   0 - 795 -      hvc0       00:00:00 init-s
0 S      0         6        2  0  80   0 - 795 -      hvc0       00:00:00 init
4 S      0        43        1  0  79  -1 - 12600 -      ?           00:00:00 system
4 S     989        78        1  0  80   0 - 5604 -      ?           00:00:00 system
4 S      0        84        1  0  80   0 - 8853 -      ?           00:00:00 system
4 S      0       104        1  0  80   0 - 722 -      ?           00:00:00 chrony
4 S      0       105        1  0  80   0 - 1118 -      ?           00:00:00 cron
4 S     995       106        1  0  80   0 - 2205 -      ?           00:00:00 dbus-d
4 S      0       122        1  0  80   0 - 11024 -      ?           00:00:00 networ
4 S      0       163        1  0  80   0 - 4609 -      ?           00:00:00 system
4 S      0       166        1  0  80   0 - 448011 -      ?           00:00:00 wsl-pr
4 S     100       186        1  0  80   0 - 55137 -      ?           00:00:00 rsyslo
4 S      0       207        1  0  80   0 - 1373 -      tty1       00:00:00 agetty
4 S     988       216       104  0  80   0 - 5900 -      ?           00:00:00 chrony
1 S     988       232       216  0  80   0 - 3024 -      ?           00:00:00 chrony
4 S      0       254        1  0  80   0 - 30865 -      ?           00:00:00 unatte
5 S      0       329        2  0  80   0 - 796 -      ?           00:00:00 Sessio
5 S      0       330       329  0  80   0 - 800 -      ?           00:00:00 Relay(
4 S    1000       331       330  0  80   0 - 1567 do_wai pts/0      00:00:00 bash
4 S      0       332        2  0  80   0 - 2162 -      ?           00:00:00 login
4 S    1000       374        1  0  80   0 - 5584 -      ?           00:00:00 system
5 S    1000       376       374  0  80   0 - 5729 -      ?           00:00:00 (sd-pa
4 S    1000       402       332  0  80   0 - 1534 poll_s pts/1      00:00:00 bash
0 R    1000       690       331  0  80   0 - 1798 -      pts/0      00:00:00 ps

```

```

gopiseti_akshay@AKSHAY:~$ top
top - 15:24:01 up 6 min, 1 user, load average: 0.02, 0.02, 0.00
Tasks: 25 total, 1 running, 24 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.
MiB Mem : 7755.7 total, 6506.2 free, 576.5 used, 825.5 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 7179.2 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM    TIME+
    1 root        20   0   24252 15424 11612 S   0.0   0.2   0:00.36
    2 root        20   0    3180  2208  2072 S   0.0   0.0   0:00.00
    6 root        20   0    3180  2048  1940 S   0.0   0.0   0:00.00
   43 root        19  -1   50400 16784 15500 S   0.0   0.2   0:00.09
   78 systemd+    20   0   22416 14480 11984 S   0.0   0.2   0:00.03
   84 root        20   0   35412 12328  9188 S   0.0   0.2   0:00.20
  104 root        20   0    2888  1952  1820 S   0.0   0.0   0:00.02
  105 root        20   0   4472  3124  2880 S   0.0   0.0   0:00.00
  106 message+   20   0   8820  5324  4652 S   0.0   0.1   0:00.03
  122 root        20   0   44096 29716 14568 S   0.0   0.4   0:00.15
  163 root        20   0   18436  9396  8156 S   0.0   0.1   0:00.05
  166 root        20   0 1792044 15332 12552 S   0.0   0.2   0:00.06
  186 syslog     20   0   220548 5756  4512 S   0.0   0.1   0:00.04
  207 root        20   0    5492  2716  2532 S   0.0   0.0   0:00.00
  216 _chrony      20   0   23600 11056  7088 S   0.0   0.1   0:00.03
  232 _chrony      20   0   12096  2464  1820 S   0.0   0.0   0:00.00
  254 root        20   0  123460 32672 18064 S   0.0   0.4   0:00.11
  329 root        20   0    3184  1112   980 S   0.0   0.0   0:00.00
  330 root        20   0    3200  1252  1108 S   0.0   0.0   0:00.02
  331 gopiset+    20   0    6268  5596  3928 S   0.0   0.1   0:00.06
  332 root        20   0    8648  5516  4672 S   0.0   0.1   0:00.00
  374 gopiset+    20   0   22336 13596 10980 S   0.0   0.2   0:00.06
  376 gopiset+    20   0   22916  4068  2068 S   0.0   0.1   0:00.00
  402 gopiset+    20   0    6136  5416  3864 S   0.0   0.1   0:00.01
  702 gopiset+    20   0    8172  5996  3888 R   0.0   0.1   0:00.00

```

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    int i;
```

```
pid_t pid;
```

```
for (i = 0; i < 3; i++)
```

```
{
```

```
    pid = fork();
```

```
    if (pid < 0)
```

```
    {
```

```
        perror("fork failed");
```

```
        exit(1);
```

```
    }
```

```
    if (pid == 0)
```

```
    {
```

```
        printf("Child %d: PID = %d, PPID = %d\n",
```

```
            i + 1, getpid(), getppid());
```

```
        sleep(i + 1);
```

```
        printf("Child %d completed.\n", i + 1);
```

```
        exit(0);
```

```
    }
```

```
}
```

```
for (i = 0; i < 3; i++)
```

```
{
```

```
    wait(NULL);
```

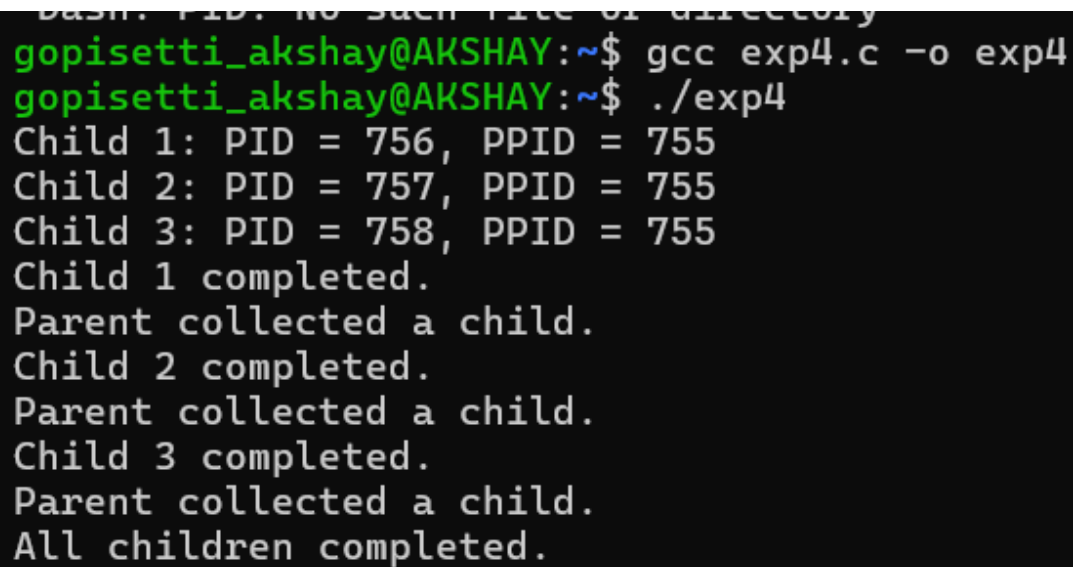
```
    printf("Parent collected a child.\n");
```

```
}
```

```
printf("All children completed.\n");
```

```
return 0;
```

```
}
```



```
bash: PID: No such file or directory
gopisetti_akshay@AKSHAY:~$ gcc exp4.c -o exp4
gopisetti_akshay@AKSHAY:~$ ./exp4
Child 1: PID = 756, PPID = 755
Child 2: PID = 757, PPID = 755
Child 3: PID = 758, PPID = 755
Child 1 completed.
Parent collected a child.
Child 2 completed.
Parent collected a child.
Child 3 completed.
Parent collected a child.
All children completed.
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
int main()
```

```
{
```

```
    pid_t pid;
```

```
    pid = fork();
```

```
    if (pid < 0)
```

```
    {
```

```
        perror("fork failed");
```

```
        return 1;
```

```
}

if (pid == 0)
{
    printf("Child process exiting...\n");
    exit(0);
}
else
{
    printf("Parent PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);

    printf("Parent sleeping for 30 seconds...\n");

    sleep(30);

    printf("Parent exiting...\n");
}

return 0;
}
```

```

gopiseti_akshay@AKSHAY:~$ gcc zombie.c -o zombie
gopiseti_akshay@AKSHAY:~$ ./zombie
Parent PID: 786
Child PID: 787
Parent sleeping for 30 seconds...
Child process exiting...
Parent exiting...
gopiseti_akshay@AKSHAY:~$ ps -el
F S  UID      PID     PPID  C  PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S   0         1         0  0  80   0 -  6063 -    ?           00:00:00 systemd
5 S   0         2         1  0  80   0 -  795 -    hvcb0       00:00:00 init-systemd(Ub
0 S   0         6         2  0  80   0 -  795 -    hvcb0       00:00:00 init
4 S   0        43         1  0  79  -1 - 12600 -    ?           00:00:00 systemd-journal
4 S  989        78         1  0  80   0 -  5604 -    ?           00:00:00 systemd-resolve
4 S   0        84         1  0  80   0 -  8853 -    ?           00:00:00 systemd-udev
4 S   0       104         1  0  80   0 -  722 -    ?           00:00:00 chronyd-starter
4 S   0       105         1  0  80   0 - 1118 -    ?           00:00:00 cron
4 S  995       106         1  0  80   0 -  2205 -    ?           00:00:00 dbus-daemon
4 S   0       122         1  0  80   0 - 11024 -    ?           00:00:00 networkd-dispat
4 S   0       163         1  0  80   0 -  4609 -    ?           00:00:00 systemd-logind
4 S   0       166         1  0  80   0 - 448011 -    ?           00:00:00 wsl-pro-service
4 S  100       186         1  0  80   0 - 55137 -    ?           00:00:00 rsyslogd
4 S   0       207         1  0  80   0 -  1373 -    tty1        00:00:00 agetty
4 S  988       216       104  0  80   0 -  5900 -    ?           00:00:00 chronyd
1 S  988       232       216  0  80   0 -  3024 -    ?           00:00:00 chronyd
4 S   0       254         1  0  80   0 - 30865 -    ?           00:00:00 unattended-upgr
5 S   0       329         2  0  80   0 -  796 -    ?           00:00:00 SessionLeader
5 S   0       330       329  0  80   0 -  800 -    ?           00:00:00 Relay(331)
4 S 1000       331       330  0  80   0 -  1567 do_wai pts/0    00:00:00 bash
4 S   0       332         2  0  80   0 -  2162 -    ?           00:00:00 login
4 S 1000       374         1  0  80   0 -  5584 -    ?           00:00:00 systemd
5 S 1000       376       374  0  80   0 -  5729 -    ?           00:00:00 (sd-pam)
4 S 1000       402       332  0  80   0 -  1534 poll_s pts/1    00:00:00 bash
0 T 1000       702       331  0  80   0 -  2043 do_sig pts/0    00:00:00 top
0 R 1000       799       331  0  80   0 -  1798 -    pts/0      00:00:00 ps
gopiseti_akshay@AKSHAY:~$ |

```

```

gopiseti_akshay@AKSHAY:~$ grep zombie
ls
hello
^Z
[2]+  Stopped                  grep --color=auto zombie
gopiseti_akshay@AKSHAY:~$ |

```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    pid_t pid;
```

```
    pid = fork();
```

```
    if (pid < 0)
```

```
    {
```

```
        perror("fork failed");
```

```

        return 1;
    }

    if (pid == 0)
    {
        printf("Child process running...\n");
        sleep(2);
        printf("Child exiting...\n");
        exit(0);
    }
    else
    {
        printf("Parent waiting for child...\n");

        wait(NULL);

        printf("Child collected. No zombie process.\n");
    }

    return 0;
}

```

```

gopisetti_akshay@AKSHAY:~$ gcc no_zombie.c -o no_zombie
gopisetti_akshay@AKSHAY:~$ ./no_zombie
Parent waiting for child...
Child process running...
Child exiting...
Child collected. No zombie process.
gopisetti_akshay@AKSHAY:~$ |

```